

AcademiCats × 小红书数据库 主计划 (MASTER PLAN)

最后更新：2026-05-13 仓库：[H:\xhs](#) 这份文档是 handoff 文档 —— 切换对话后，把这个文件 + 仓库一起喂给新 Claude 就能继续。

0. 项目使命

AcademiCats 是面向下沉用户的 AI 学术工作台 —— **一站式一条龙快捷服务**：检索 → 引用 → AI 代写框架 → 查重去重 → 审核 → 修改。

产品定位

- **不是给学术大牛的精修工具**，是给**下沉学生群体**的“快捷救命包”
- **不假装自己是研究助手**：AI 实质就是代写引擎，包装为「引导式论文生成框架」
- **用户名义上主导内容**，实际上 AI 出绝大部分文字、用户负责定方向 + 验收
- **核心竞争力 = 速度 + 一条龙**（不是质量、不是学术深度）

核心服务链（也就是产品功能模块）

1. **多资源平台快速文献检索（十几秒级）** ★ 核心 USP - 一次输入，并发查询 知网 / Web of Science / Scopus / PubMed / Google Scholar 等 - 秒级返回（传统跨库手动查 → 几十分钟） - 给下沉用户：**不用再翻 5 个数据库**
2. **一键引用** - 文献搜到直接插入正文，自动生成 citation (APA / MLA / Chicago / GB/T-7714) - 引用与参考文献列表自动同步
3. **引导式论文框架生成 (= AI 代写)** - 主题输入 → AI 给大纲 → AI 填章节内容 → 用户调方向 - **主打“一条龙”**：用户不需要一段段写，AI 跑通整个流程
4. **AI 查重 / 去重 / 编辑** - 重复率检测（对标知网 / Turnitin） - AI 改写降重（核心刚需，下沉用户必踩坑） - 语言润色
5. **文章审核（自审稿）** - AI 用同行评议视角看：逻辑链 / 论证 / 引用充分性 / 语法
6. **重新修改 / 迭代** - 基于审核反馈 + 用户调整 → 下一轮 AI 重写部分段落 - 整个循环：用户主导方向，AI 主导文字劳动

目标用户画像

- **主要**：海外留学生（“留子”）写 essay / paper / assignment（赶 ddl，要快）
- **同等重要**：国内本科 / 研究生写毕业论文 / 开题报告（降重痛）
- **边缘**：博士 / 期刊投稿（这群人不是我们的下沉受众）

下沉特征：- 不挑剔学术质量，要的是“交得上去 + 不被查重” - 价格敏感 - 决策点：能不能省时间、能不能避开降 AI 率 / 查重坑

为什么爬小红书

xhs 是下沉学生**痛点最集中、表达最直白**的平台。爬来的数据用于：

- 1. **产品 feature 优先级** - 评论里最高频的"求 prompt 模板"、"求工具链接"、"怎么降 AI 率" - → 反推：用户买产品时最想要什么
- 2. **竞品分析** - 同赛道有谁（chatgpt 写论文、DeepSeek 写论文、各类 AI 代写工具） - 他们被用户夸什么、骂什么 - 我们怎么差异化（**速度 + 一条龙** 是我们的牌）
- 3. **冷启动起号策略** - 哪个 niche CR 低（论文降重？查重避坑？文献综述？） - 什么内容形式 ROI 高（图文 vs 视频、教程 vs 测评） - 多久发一次、配什么 hashtag
- 4. **营销内容参考** - 爆款标题 hook 结构（数字 / 痛点 / 故事） - 高赞 note 的内容骨架（问题 → 痛点 → 工具 → 步骤 → 效果） - 转化点（评论里被问"求私信"、"求链接"的内容形态）

1. 当前数据库快照

data/xhs.db (SQLite, WAL mode)

指标	数量
总 notes	429
图文 note	397
视频 note	32
实质 body (>100 字符)	307
评论总数 (已抓)	1,497
有评论的 note	98 / 429
图片数	1,624 (全本地)
不同作者	376
总点赞	726,945
总收藏	670,270
总评论 (xhs 上报)	94,051
最热 note	「关于我一个学期写四篇文献综述」34K 赞

已覆盖关键词（每个 ~20-42 notes）： 研究生日常 / 文献综述 / 开题报告 / 论文写作 / 文献阅读 / 科研工具 / 留学PS / 论文降重 / 毕业论文 / ChatGPT写论文 / 论文润色 / AI写论文

队列状态： done=429, error=48, pending=0（队列已空）

2. DB Schema

notes (主表)

```
note_id (PK), xsec_token, url, type, title, body,  
author_id, author_nickname, publish_time_ms, ip_location,  
liked_count, collected_count, comment_count, share_count, image_count,  
video_url, video_duration_ms,  
tags_json, at_users_json, raw_json,  
crawled_at, updated_at
```

comments

```
comment_id (PK), note_id, parent_id, user_id, nickname,  
content, like_count, sub_comment_count, publish_time_ms,  
ip_location, pictures_json, raw_json, crawled_at
```

authors

```
user_id (PK), nickname, red_id, avatar, description, gender,  
ip_location, fans_count, follows_count, notes_count, interaction_count
```

images

```
note_id, idx, url, width, height, local_path, downloaded_at  
PRIMARY KEY (note_id, idx)
```

discover_queue (爬虫队列)

```
note_id (PK), xsec_token, xsec_source,  
source_type ('search'|'author'|'topic'|'url'),  
source_value (the keyword / user_id),  
status ('pending'|'in_progress'|'done'|'error'),  
attempts, last_error, discovered_at, last_attempt_at
```

crawl_log (事件日志)

```
id, ts, level, event, detail
```

3. 技术栈与关键突破

3.1 反爬突破（关键！）

xhs 的反爬层叠： 1. IP throttle（同 IP 高频访问 → 300013 redirect） 2. TLS / JA3 fingerprint（plain httpx 直接被 reject） 3. Browser fingerprint（无头 Chrome 被识别） 4. X-s / X-t signature（所有 API 调用必须签） 5. 账号权限（新号 / 买号没 API 权限） 6. xsec_token 跟 a1 绑定（token 复用会跳登录）

我们的破解方法： - `curl_cffi` + `impersonate="chrome131"` —— TLS 指纹伪装成真 Chrome，0 风控、绕过 JA3 - SSR HTML 提取 `window.__INITIAL_STATE__` —— 不调 API，直接拿 SSR 渲染好的 note 数据 - 安全 stringify JS —— 应对 Vue 反应式对象的循环引用 - 原子队列 claim（SQLite `UPDATE ... RETURNING`） —— 多 worker 并发安全 - DrissionPage 真浏览器 + XHR 监听 —— 用于 Phase 1/2 discover 阶段

3.2 性能记录

- `curl_cffi` http-drain: **195 notes in 4 min**, 0 rate-limited
- DrissionPage discover: ~30-40 notes / keyword / 5 min

3.3 Python 依赖（已装在 `.venv/`）

```
curl_cffi      # chrome131 TLS 伪装
DrissionPage   # 真浏览器 + CDP
loguru         # 日志
xhshow         # xhs API 签名（备用，新号没权限可用）
openpyxl       # 读外包 Excel
pycryptodome   # AES（解 Chrome cookie，已弃用因 ABE）
pywin32        # DPAPI（同上）
```

4. 文件清单 (核心代码)

`crawler/`（核心爬虫库）

- `config.py` — 路径常量、API 端点模式、pacing 参数
- `db.py` — SQLite DAO, `upsert_note`、`upsert_comments`、`claim_queue_items`、`release_claim`
- `browser.py` — DrissionPage 封装, network listener、`read_initial_state`、风控检测
- `discover.py` — 4 种 discover: search / author / topic / urls
- `detail.py` — DrissionPage 详情抓取（慢但能抓评论 via XHR 监听）
- `httpx_detail.py` — `curl_cffi` 快速详情抓取（生产主力, 50-100 notes/min）
- `parse.py` — `__INITIAL_STATE__` 解析、camelCase ↔ snake_case 兼容
- `login.py` — DrissionPage 登录流程（带 force re-scan）
- `main.py` — CLI 入口, `python -m crawler.main {search,detail,http-drain,...}`
- `export.py` — JSON / CSV 导出

tools/ (独立脚本工具)

生产工具： - `login_extract.py` — 弹 Chrome 扫码登录，提取所有 cookies 到 `state/xhs_full_cookies.json` - `render_dashboard.py` — 渲染单文件 HTML 看板 (GitHub Pages 兼容, no-referrer) - `build_analysis_package.py` — 拼接所有 notes + 评论 + 产品上下文成单个 .md 文件喂给 LLM - `auto_refresh.py` — 定时刷新看板 + git push 的 daemon - `phase3.py` — Phase 3 关键词扩展编排 (等队列空后自动拓展)

诊断 / 一次性工具 (保留作参考, 多数已废) : - `test_browser_cookies.py` — 验证 Chrome cookie 提取 (被 ABE 干掉) - `test_anonymous.py` — 验证匿名 curl_cffi 路径 - `test_signed_search.py` / `test_comment_api.py` / `test_feed_api.py` / `test_feed_refresh.py` / `test_feed_full.py` — 签名 API 端点测试, 确认账号权限 - `test_explore_bare.py` — 测试 bare URL / pc_feed / discovery/item 变体 - `test_ssr_with_login.py` — 登录态 SSR 是否含评论 (否) - `test_anon_search.py` / `test_anon_signed_search.py` — 匿名搜索测试 - `test_topic.py` / `test_author_profile.py` / `test_author_with_login.py` / `test_warmed_search.py` — 各种 discovery 路径测试 - `comments_browser.py` — DrissionPage XHR 监听抓评论 (受限于账号权限) - `discover_authors_ssr.py` — 匿名 author profile SSR 抓评 (验证为空) - `dump_search_html.py` / `debug_author_structure.py` — 结构 dump - `check_missing.py` / `check_coverage.py` / `snapshot.py` / `rank_authors.py` / `filter_excel.py` / `inspect_excel.py` — 数据分析查询

辅助工具： - `apify_run.py` — Apify zhorex actor 调用包装 - `proxy_chain.py` — 本地代理链 (用过 Decodo 国内住宅代理, 已不用) - `warmup.py` — Chrome profile 烘热脚本 - `import_chrome_cookies.py` — 从用户 Chrome 导入 cookies (被 ABE 干掉) - `open_chrome_manual.py` — 打开 Chrome 调试入口

state/ (运行时数据)

- `xhs_full_cookies.json` — 登录后所有 cookies (含 a1 + web_session)
- `profile/`, `profile-w1..w7`, `profile-w99/` — Chrome profile (per worker)
- `profile-login/` — QR 扫码登录用的 profile
- `relevant_authors.txt` — 留子-写论文相关作者 ID + 昵称
- `excel_relevant_urls.txt` — Excel 外包筛过的 URL (艺术留学, 跟 AcademiCats 不直接相关)
- `phase4_discover.log` — Phase 4 尝试日志
- `comments_browser.log` — 浏览器抓评论尝试日志

data/

- `xhs.db` — 主 SQLite
 - `images/<note_id>/<id>.{jpg,png,webp}` — 原始图片
 - `exports/`
 - `dashboard.html` — 看板 (push 到 GitHub Pages)
 - `xhs_analysis_package.md` — LLM 分析包 (646KB, 全部 notes + 评论 + 产品上下文)
 - `apify/<run_id>.json` — Apify 历史 run 结果
 - `外包_任务3000.xlsx` — 外包数据 (2891 行, 艺术留学主题, 与 AcademiCats 不直接对齐)
-

5. 2026-05 xhs 反爬升级 (重要)

xhs 这一两周大规模收紧 web 端 anti-bot。所有匿名 + 弱账号路径都被堵了：

路径	之前状态	现状 (2026-05-13)
匿名 <code>/explore/<id></code> SSR	✅ 直接拿 note 详情	✅ 依然能用 (curl_cffi chrome131)
匿名 <code>/search_result?keyword=</code> SSR	✅ feeds 数组有数据	❌ feeds 是空数组
匿名 <code>/search/notes</code> XHR	(浏览器自动调) ✅	❌ -101 / 不再 fire
匿名 <code>/search/recommend</code>	n/a	❌ -101 no login
匿名 topic SSR (<code>type=54</code>)	✅	❌ feeds 空
匿名 <code>/user/profile/<uid></code> SSR	✅ user.notes 有内容	❌ user.notes 是空骨架
新号登录态 <code>/search/notes</code> (signed)	n/a	❌ -104 no permission
新号登录态 <code>/comment/page</code> (signed)	n/a	❌ 461 stealth-fail (body 假装成功, data 空)
新号登录态 <code>/feed</code> (signed)	n/a	⚠️ 起初 200 OK 后期 461 (被风控 ramp-up)
新号登录态 <code>/explore/<id></code> 带旧 token	n/a	❌ 302 跳登录 (token 跟新 a1 不绑)

关键洞察：xsec_token 是跟 a1 绑死的。我们 DB 里的 token 是匿名 a1 时期抓的，跟新登录的 a1 不兼容，所以用登录账号去补漏漏掉的内容这条路走不通。

唯一未试的路径：用户本人长期使用的真实账号（3 个月+ 历史，有真实浏览/点赞/关注/发布）。这种账号 xhs 会授予完整 API 权限。

6. 未来扩展路径 (Path A → E)

🏆 Path A: 用户老号登录 → 全方位补齐 (推荐)

条件：用户用自己日常用的 xhs 账号扫码（不能是新注册、不能是买的）。

能解锁： - ✅ `/search/notes` signed API → 任意关键词搜索 + fresh xsec_token - ✅ `/comment/page` signed API → 补 92K+ 漏掉的评论 - ✅ `/feed` signed API → 刷新现有 429 notes 的 like/collect/comment 计数 - ✅ `/user_posted` → 批量抓单个作者的全部 notes（最多~200 per author）

预计产出：1500-3000 新 notes + 现有 notes 评论补齐 + 实时计数更新

执行步骤：1. 删 `state/profile-login/`，重跑 `python tools/login_extract.py` 2. 用户扫码（用老号）3. 调一次 `python tools/test_comment_api.py` 验证（要看到 code 0 + 真实评论）4. 验证通过则：- 写 `tools/signed_search.py`（curl_cffi + xhshow，迭代 30 keywords × 10 pages）- 写 `tools/comments_drain.py`（curl_cffi + xhshow，遍历 DB 补评论）- 同时跑 `python -m crawler.main http-drain --concurrency 8`（处理新加入队列）

🏆 Path B: 现有 429 notes 深度分析 (即可启动)

条件: 不需新数据。

可执行分析: 1. **标题 pattern 挖掘:** - 数字开头 ("一个学期写四篇...")、emoji 用法、痛点句式、问句钩子 - 跑 LLM 把 top-50 高赞 note 的标题归类: 故事型 / 教程型 / 痛点型 / 工具型 2. **内容结构分析:** - 字数 vs 互动率 回归 - 首段 hook (前 50 字) 的共性 - 图片数量 distribution (图文 note 普遍 6-9 图) 3. **发布时机:** - publish_time_ms histogram by hour-of-day, day-of-week 4. **Hashtag 共现网络:** - 解析 `tags_json`, 统计共现, 找"留子写论文"周围的 hashtag 集群 5. **评论需求挖掘:** - 用 LLM 把 1497 条评论分类 (提问 / 求资源 / 同感 / 推荐) - 提取"求 prompt"、"求工具链接"等高频询问 → 这是 AcademiCats 产品 feature backlog 6. **作者类型分析:** - 376 作者按 (notes_count, avg_likes, niche) 聚类 - 找"留子写论文"赛道 top 10 KOL → 复制其 angle 7. **爆款公式:** - 跑 LLM 对比 top-20 vs bottom-20 by likes, 提炼 "what makes a note pop"

用 `data/exports/xhs_analysis_package.md` (已经存在, 646KB) 直接喂给 GPT-5 / Claude Opus 出报告。

🏆 Path C: 多号矩阵长期项目

条件: 注册 5-10 个真实账号, 每个累积 1-2 周浏览/互动历史。

执行: - 用 SIM 卡多注册真实号 - 每个号在真实手机上日均 20 分钟浏览 / 互动 (点赞 + 评论 + 关注) - 一周后账号成熟, 分发给 5 个 Chrome profile - 用 multi-worker (`worker_id` 0-9) 并发抓取, 每号 / 每 IP 限速 - 预计产出: 10K+ notes / month sustained

缺点: 时间成本高, 规模化难。

Path D: Apify 第三方付费

条件: 付费。

`tools/apify_run.py` 已经能调用 `zhorex/rednote-xiaohongshu-scraper` actor: - 1000 notes ≈ ¥100 RMB - 评论按页计费 (10 条评论 ≈ ¥0.08) - 快但贵, 并且 xhs 反爬升级后 Apify actor 也越来越不稳定
之前测试: easyapi actor broken; zhorex actor 部分功能被 xhs 屏蔽。

Path E: MediaCrawler (NanmiCoder) 集成

条件: 1.6K star 开源项目, 比我们的更成熟。

特点: - Playwright + signed API + 风控规避 (更先进) - 但同样需要登录账号才能跑评论 / 搜索 - 适合作为 Path A 的备份实现

仓库: <https://github.com/NanmiCoder/MediaCrawler>

7. 关键词矩阵 (按 AcademiCats 产品功能对齐)

重新分级: 跟「产品功能模块」——对应, 不是按"留子写论文"这一个角度。下沉用户、一站式、速度、AI 代写、查重去重 = 我们要爬的全部主题。

Tier 1A — 文献检索 (核心 USP, 必抓)

文献检索	文献查找	知网检索	知网技巧
Google Scholar	Web of Science	Scopus 检索	PubMed
文献调研	文献搜集	中外文文献	文献查询工具
英文文献查找	SCI检索	核心期刊检索	文献神器

Tier 1B — 引用 / 参考文献 (核心功能)

一键引用	Citation	参考文献格式	APA格式
MLA格式	Chicago	GB7714	EndNote引用
Zotero引用	文献管理	引用工具	参考文献神器

Tier 1C — AI 代写 / 论文生成 (核心功能, 最大流量赛道)

AI写论文	ChatGPT写论文	DeepSeek写论文	Claude写论文
Gemini写论文	Kimi写论文	豆包写论文	文心一言写论文
通义千问写论文	讯飞星火写论文	秘塔AI	橙篇
AI论文神器	论文一站式	论文生成	论文代写
开题报告生成	文献综述生成	论文初稿	论文框架

Tier 1D — 查重 / 降重 / AI 检测 (核心刚需, 下沉痛点)

论文查重	知网查重	万方查重	维普查重
PaperPass	论文降重	降重技巧	降AI率
AI检测	AI率	AI痕迹	学术不端
查重避坑	Turnitin	格子达	论文检测

Tier 2 — 论文流程辅助 (高相关)

论文润色	语法润色	学术英语	学术翻译
论文图表	SCI写作	论文修改	论文修订
答辩	答辩PPT	答辩准备	审稿
论文审核	同行评议	论文逻辑	论文结构

Tier 3 — 用户场景与痛点 (场景关键词)

留学生写论文	留学生作业	留学生essay	留学生assignment
留学生paper	赶ddl	ddl救命	熬夜写论文
毕业论文	毕业季	本科毕业论文	研究生论文
研究生日常	博士日常	论文焦虑	论文拖延

Tier 4 — 工具生态（竞品 / 周边）

Zotero	EndNote	Notion AI	Obsidian论文
飞书文档	WPS AI	Word论文模板	LaTeX
ScienceWeaver	秘塔搜索	Perplexity	Felo

Tier 5 — 边缘 / 长尾（按需）

留学PS	留学CV	推荐信
留子日常	留学党	海归
考研日常	期末复习	学习方法

抓取策略（调整后）： - Tier 1A-D： **必爬，每个 keyword 3 种排序（general + popularity + time）**，每排序 15-20 页 → 每关键词 ~40-60 unique notes - Tier 2： popularity + time，每关键词 ~30-40 unique - Tier 3： popularity only，每关键词 ~20-30 - Tier 4： popularity only，每关键词 ~10-20 - Tier 5： 低优，按需

目标产出： - Tier 1A-D 总约 70 个关键词 × 3 排序 × ~50 notes ≈ **~6000-8000 unique notes**（去重后 ~3000） - 加 Tier 2-5： **总 5000-8000 unique notes**

核心爆款来源（基于当前 429 notes 观察）： - 「DeepSeek 喂饭指令 | 一天完成论文初稿」类（24K 赞） → AI 代写 - 「关于我一个学期写四篇文献综述」类（34K 赞） → 一条龙 - 「两小时就写完了开题报告」类（24K 赞） → 速度 - 「博士学姐分享 | 文献综述的写作方法」类（16K 赞） → 教程 - 上述全部跟我们产品 USP 直接对齐

8. 后期工具（用爬到的数据做什么）

两个完全不同的用途： - **8A**： 辅助 AcademiCats 产品本身（功能优先级、定价、产品 copy） - **8B**： 辅助 AcademiCats 的小红书起号（营销、获客）

8A. 反哺产品本身

- 1. **功能优先级 backlog** - 评论高频问题 / 抱怨 → 用户最痛的点 - 例：评论里反复出现"求降 AI 率方法" → 降重功能要做到极致 + 在产品 onboarding 顶上 - 评论里抱怨"知网查重太贵" → 我们提供平替 / 一站式必须算上查重
- 2. **竞品监控** - 爬"AI写论文 + 各品牌"关键词，看每家被夸什么 / 骂什么 - 写「竞品 ATSWOT 表」： 速度 / 一站式 / 价格 / 查重 / 引用支持 - 我们怎么定位： **速度 + 一条龙**（市面上没人同时做这两点）
- 3. **下沉用户语言** - 爬到的标题、正文、评论 → 用户真实说话方式 - 产品文案 / onboarding 语气直接抄过来（不要写"学术工作台"，要写"AI 一键搞定毕业论文"）
- 4. **定价信号挖掘** - 评论里"某某工具 9.9 / 30 / 198 元" → 用户接受的价格带 - "划算 / 太贵 / 收费看不到" → 定价策略和展示方式
- 5. **关键 use case 整理** - 把高赞 note 的真实工作流 cluster 成 5-10 个标准 use case - 每个 use case 是产品的一个 demo 场景

8B. 小红书起号 + 内容矩阵

1. **赛道选择** - 基于 DB 的 author × engagement 聚类，找当前 CR (competition ratio) 最低、平均互动最高的细分赛道 - 例如可能不是"AI 写论文" (已红海) 而是"知网查重避坑"
2. **爆款公式** - 跑 LLM 对比 top-20 vs bottom-20 by likes (同主题)，提炼 hook / 节奏 / 字数 / 配图模式 - 输出「AcademiCats 起号 SOP」文档
3. **内容日历** - 基于 publish_time 分布 → 一周 5 帖建议时间表 - 配套：每帖给 prompt 模板 (不是 AI 全写，给 hook + 结构 + 关键词)
4. **互动 / 评论运营** - 高频问题清单 → 评论自动回复 / 半自动回复模板 - 评论筛选优先级 (按 like_count + 是否含转化意图，如"求私信")
5. **冷启动 100 帖规划** - 30 帖工具教程型 ("DeepSeek 写论文 prompt 大全") - 30 帖痛点共鸣型 ("赶 ddl 一晚上写完论文") - 30 帖产品 demo 型 (明面是教程，暗里是 AcademiCats 入口) - 10 帖互动型 (投票 / 提问 / 求建议)

8C. 数据基础设施

- **持续爬取**：cron job 每周补 30-50 新 notes + 刷新现有 note 计数
- **向量化语料**：notes + comments embed 到 vector DB (pgvector / weaviate)
- 支持产品端 RAG (用户问"怎么降重" → 检索高赞 xhs 教程 → 生成答案)
- 支持内部分析 (用语义搜索找特定 cluster)
- **标注**：人工标 100 篇为"AcademiCats 高相关 / 中 / 低"，训练分类器自动筛后续 raw data
- **关键词扩词系统**：从评论里挖新的高频长尾词 → 反馈到爬取关键词矩阵

9. 切换对话后的接续 Prompt 模板

把这个直接复制给新 Claude：

我有一个本地项目 H:\xhs，是为我自己的 AI 学术工作台产品 AcademiCats 爬小红书数据用的。

【背景与计划】请先 cat 或读 H:\xhs\MASTER_PLAN.md，里面有完整产品方向、当前数据库快照、技术架构、

【当前状态】429 notes / 1497 comments / 376 authors 在 data\xhs.db。看板在 data/exports/dashboard

【我现在想做】XXX (填具体任务)

【约束】

- Windows 10 + PowerShell + .venv (已装 curl_cffi, DrissionPage, xhshow 等)
- 不要碰 state/profile-login/ (里面是当前登录态)
- 看板会自动推到 GitHub Pages，注意 no-referrer + https URL

【可执行】请先看 MASTER_PLAN.md 的"未来扩展路径"小节，我们应该走 Path X。

10. 紧急 / 常用命令速查

```
# 看 DB 状态
.venv\Scripts\python.exe tools\snapshot.py

# 看现有覆盖
.venv\Scripts\python.exe tools\check_coverage.py

# 跑 detail drain (curl_cffi, 处理 pending 队列)
.venv\Scripts\python.exe -m crawler.main http-drain --concurrency 8

# 跑 detail (browser, 用于评论)
.venv\Scripts\python.exe -m crawler.main detail --comment-pages 5

# 弹 Chrome 扫码登录
.venv\Scripts\python.exe tools\login_extract.py

# 渲染看板
.venv\Scripts\python.exe -m crawler.main dashboard

# 重建分析包
.venv\Scripts\python.exe tools\build_analysis_package.py

# 关键词搜索 discover
.venv\Scripts\python.exe -m crawler.main search --keywords "AI写论文,论文写作" --pages 15 --sort

# 看队列里 pending 多少
.venv\Scripts\python.exe -c "import sqlite3; print(sqlite3.connect('data/xhs.db').execute(\"SELEC
```

11. 已知遗留问题 / TODO

- 48 个 queue 失败的 note — `discover_queue WHERE status='error'`，主要因 token 失效或 deleted。可重试 with `--retry-errors` 但价值不大。
- 321 / 429 notes 评论数 = 0 — 不是没评论，是 xhs SSR 不渲染。需 Path A 老号补齐。
- 图片有部分缺失 — 个别 note 的 `image_list` 是空，因 SSR 截断或图片域名换了。约 5-10 个 note 受影响。
- dashboard.html 引用 xhs CDN 图片 — 用 `<meta name="referrer" content="no-referrer">` 绕过 referrer 检查。GitHub Pages 上能显示。
- `xhs_analysis_package.md` 是手动 rebuild 的 — 想自动化要写个 cron。

12. 一句话总结

AcademiCats = 面向下沉学生群体的一站式 AI 论文一条龙服务（检索 → 引用 → AI 代写 → 查重去重 → 审核 → 修改）。已建：429 notes 高质量学术写作 + AI 代写赛道语料 + 完整爬取基础设施。已堵：xhs 2026-05 大幅升级反爬，所有匿名 + 新号路径全死。下一步：用户用真实老号登录解锁 signed API (Path A)，或就用现有数据出深度分析 (Path B)。